CP MASTER ALGORITHM REFERENCE

WHAT TO USE + WHEN TO USE + COMPLEXITY + COPY-PASTE TEMPLATES

This is a CP revision/reference sheet.

Do not memorize templates blindly. Learn the problem pattern, invariant,

complexity, indexing convention, and edge cases.

1. PREFIX / SUFFIX TECHNIQUES

PREFIX SUM

WHEN:

• Many range-sum queries.

• Need sum of [l,r] repeatedly.

TIME: O(n) build, O(1) query.

```
vector<ll> pref(n+1);
for(int i=0;i<n;i++) pref[i+1]=pref[i]+a[i];
ll sum=pref[r+1]-pref[l];
```

SUFFIX SUM

WHEN:

• Need sum from i to n-1.

• Split array into left/right parts.

• Prefix + suffix answer problems.

TIME: O(n).

```
vector<ll> suff(n+1);
for(int i=n-1;i>=0;i--) suff[i]=suff[i+1]+a[i];
```

PREFIX/SUFFIX MIN/MAX

WHEN:

• Need best value on either side of every index.

• Split-array greedy problems.

TIME: O(n).

```
vector<ll> prefMax(n),suffMax(n);
prefMax[0]=a[0];
for(int i=1;i<n;i++) prefMax[i]=max(prefMax[i-1],a[i]);
suffMax[n-1]=a[n-1];
for(int i=n-2;i>=0;i--) suffMax[i]=max(suffMax[i+1],a[i]);
```

PREFIX/SUFFIX GCD

WHEN:

• GCD after removing one element.

• GCD of left/right sections.

```
vector<ll> prefGcd(n+1),suffGcd(n+1);
for(int i=0;i<n;i++) prefGcd[i+1]=gcd(prefGcd[i],a[i]);
for(int i=n-1;i>=0;i--) suffGcd[i]=gcd(suffGcd[i+1],a[i]);
ll g=gcd(prefGcd[i],suffGcd[i+1]);
```

PREFIX/SUFFIX XOR

WHEN:

• Range XOR.

• XOR after removing/splitting.

• Bitwise array problems.

```
vector<ll> px(n+1);
for(int i=0;i<n;i++) px[i+1]=px[i]^a[i];
ll x=px[r+1]^px[l];
```

DIFFERENCE ARRAY

WHEN:

• Many range additions and only final values are required.

TIME: O(1) update, O(n) reconstruction.

```
vector<ll> diff(n+1);
diff[l]+=x;
diff[r+1]-=x;
for(int i=1;i<n;i++) diff[i]+=diff[i-1];
```

2D PREFIX SUM

WHEN:

• Many rectangle/submatrix sum queries.

TIME: O(nm) build, O(1) query.

```
vector<vector<ll>> p(n+1,vector<ll>(m+1));
for(int i=1;i<=n;i++)
 for(int j=1;j<=m;j++)
  p[i][j]=a[i-1][j-1]+p[i-1][j]+p[i][j-1]-p[i-1][j-1];
ll sum=p[x2][y2]-p[x1-1][y2]-p[x2][y1-1]+p[x1-1][y1-1];
```

2. SLIDING WINDOW / TWO POINTERS

FIXED SLIDING WINDOW

WHEN:

• Every window has exactly k elements.

• Sum/min/max/count of every length-k subarray.

VARIABLE SLIDING WINDOW

WHEN:

• Longest/shortest valid subarray.

• Expand right, shrink left while invalid.

• Works when the condition has the required monotonicity.

```
int l=0;
for(int r=0;r<n;r++){
    // add a[r]
    while(/* invalid */){
        // remove a[l]
        l++;
    }
    // [l,r] valid
}
```

TWO POINTERS

WHEN:

• Sorted array and pair condition.

• Find pair with target sum.

• Merge-like scans.

• Remove O(n^2) brute force when pointer movement is monotonic.

```
int l=0,r=n-1;
while(l<r){
    ll s=a[l]+a[r];
    if(s==target){ /* found */ l++; r--; }
    else if(s<target) l++;
    else r--;
}
```

KADANE

WHEN:

• Maximum sum contiguous subarray.

TIME: O(n).

```
ll cur=0,ans=LLONG_MIN;
for(ll x:a){
    cur=max(x,cur+x);
    ans=max(ans,cur);
}
```

3. STACK / QUEUE / DEQUE

STACK

WHEN:

• Matching brackets.

• Undo/LIFO behavior.

• Expression parsing.

• DFS-style iterative processing.

```
stack<int> st;
st.push(x); st.top(); st.pop();
```

QUEUE

WHEN:

• BFS.

• First-in-first-out processing.

• Level-order traversal.

```
queue<int> q;
q.push(x); q.front(); q.pop();
```

DEQUE

WHEN:

• Need both-end operations.

• Sliding windows.

• Monotonic queue.

MONOTONIC STACK - NEXT GREATER

WHEN:

• Next greater element.

• Previous greater.

• Stock span.

• Histogram.

• Contribution problems.

```
vector<int> nge(n,-1);
stack<int> st;
for(int i=n-1;i>=0;i--){
    while(!st.empty() && a[st.top()]<=a[i]) st.pop();
    if(!st.empty()) nge[i]=a[st.top()];
    st.push(i);
}
```

MONOTONIC STACK - NEXT SMALLER

WHEN:

• Next/previous smaller.

• Histogram.

• Minimum contribution.

```
vector<int> nse(n,-1);
stack<int> st;
for(int i=n-1;i>=0;i--){
    while(!st.empty() && a[st.top()]>=a[i]) st.pop();
    if(!st.empty()) nse[i]=a[st.top()];
    st.push(i);
}
```

MONOTONIC QUEUE

WHEN:

• Maximum/minimum of every window of size k.

TIME: O(n).

```
deque<int> dq;
for(int i=0;i<n;i++){
    while(!dq.empty() && dq.front()<=i-k) dq.pop_front();
    while(!dq.empty() && a[dq.back()]<=a[i]) dq.pop_back();
    dq.push_back(i);
    if(i>=k-1) cout<<a[dq.front()]<<'\n';
}
```

4. LINKED LIST

REVERSE LINKED LIST

WHEN:

• Reverse whole list or use reversal as part of a larger solution.

```
ListNode* prev=nullptr;
ListNode* cur=head;
while(cur){
    ListNode* nxt=cur->next;
    cur->next=prev;
    prev=cur;
    cur=nxt;
}
head=prev;
```

FAST/SLOW POINTER

WHEN:

• Find middle.

• Detect cycle.

• Find cycle entry.

• Palindrome linked list.

```
ListNode *slow=head,*fast=head;
while(fast && fast->next){
    slow=slow->next;
    fast=fast->next->next;
}
```

5. SORTING ALGORITHMS

STL SORT

WHEN:

• Almost always the default choice.

• Need sorted order.

TIME: O(n log n).

```
sort(a.begin(),a.end());
```

SORT DESCENDING

```
sort(a.rbegin(),a.rend());
```

CUSTOM COMPARATOR

WHEN:

• Sort by multiple fields.

• Intervals, pairs, objects.

• Need special ordering.

```
sort(v.begin(),v.end(),[](auto &x,auto &y){
    if(x.first!=y.first) return x.first<y.first;
    return x.second>y.second;
});
```

COUNTING SORT

WHEN:

• Values are integers in a SMALL range.

• Need O(n + range).

• Do NOT use when range is huge.

```
vector<int> cnt(MAXV+1);
for(int x:a) cnt[x]++;
for(int x=0;x<=MAXV;x++)
    while(cnt[x]--) cout<<x<<" ";
```

RADIX SORT

WHEN:

• Huge number of integers with manageable digit/bit processing.

• Specialized high-performance integer sorting.

• Rarely necessary in normal CF.

MERGE SORT

WHEN:

• Need stable sorting.

• Need inversion counting.

• Need divide-and-conquer merge logic.

TIME: O(n log n).

INVERSION COUNT

WHEN:

• Count pairs i<j with a[i]>a[j].

• Use merge sort or Fenwick + compression.

TIME: O(n log n).

```
// Standard approach: merge sort while counting cross inversions.
```

PARTIAL SORT / TOP K

WHEN:

• Only need k smallest/largest, not complete sorting.

• Consider heap, nth_element.

TIME:

nth_element average O(n), heap O(n log k).

6. HEAP / PRIORITY QUEUE

MAX HEAP

WHEN:

• Repeatedly need current maximum.

• Greedy selection of largest item.

```
priority_queue<int> pq;
pq.push(x);
int mx=pq.top();
pq.pop();
```

MIN HEAP

WHEN:

• Repeatedly need current minimum.

• Greedy selection of smallest item.

• Dijkstra.

• Scheduling.

```
priority_queue<int,vector<int>,greater<int>> pq;
```

HEAP FOR TOP K

WHEN:

• Find k largest/smallest.

• Maintain only k candidates.

• Streaming data.

Typical idea:

• k largest -> min heap of size k.

• k smallest -> max heap of size k.

TWO HEAPS

WHEN:

• Dynamic median.

• Maintain lower half and upper half.

CUSTOM HEAP

WHEN:

• Priority depends on multiple fields.

• Store pair/tuple and use custom comparator.

Dijkstra uses a MIN HEAP:

WHEN:

• Graph has NON-NEGATIVE edge weights.

• Need single-source shortest paths.

7. BINARY SEARCH

NORMAL BINARY SEARCH

WHEN:

• Sorted array.

• Search exact element or boundary.

```
int l=0,r=n-1;
while(l<=r){
    int mid=l+(r-l)/2;
    if(a[mid]==x) break;
    if(a[mid]<x) l=mid+1;
    else r=mid-1;
}
```

LOWER_BOUND

WHEN:

• First position with value >= x.

```
auto it=lower_bound(a.begin(),a.end(),x);
```

UPPER_BOUND

WHEN:

• First position with value > x.

```
auto it=upper_bound(a.begin(),a.end(),x);
```

BINARY SEARCH ON ANSWER

WHEN:

• Answer is numeric.

• Can define monotonic check(mid).

• "Minimum possible maximum" / "maximum possible minimum".

Minimum valid:

```
ll lo=low,hi=high;
while(lo<hi){
    ll mid=lo+(hi-lo)/2;
    if(check(mid)) hi=mid;
    else lo=mid+1;
}
```

Maximum valid:

```
ll lo=low,hi=high;
while(lo<hi){
    ll mid=lo+(hi-lo+1)/2;
    if(check(mid)) lo=mid;
    else hi=mid-1;
}
```

8. HASHING / MAPS

UNORDERED_MAP

WHEN:

• Frequency/count/lookup by key.

• Average O(1) operations.

• Be aware of adversarial/hash issues.

```
unordered_map<ll,int> mp;
mp[x]++;
```

MAP

WHEN:

• Need keys sorted.

• Need lower_bound/upper_bound.

TIME: O(log n).

SET

WHEN:

• Unique values in sorted order.

• Dynamic min/max or boundaries.

MULTISET

WHEN:

• Sorted values with duplicates.

• Dynamic min/max.

• Sliding-window deletion.

9. TRIE / STRING

TRIE

WHEN:

• Prefix queries.

• Dictionary of words.

• Insert/search strings.

• Prefix counts.

BINARY TRIE

WHEN:

• Maximum/minimum XOR.

• Bitwise integer queries.

KMP / PREFIX FUNCTION

WHEN:

• Pattern matching.

• Prefix-suffix/border problems.

• String periodicity.

TIME: O(n).

Z ALGORITHM

WHEN:

• Pattern matching.

• Need longest prefix match starting at every position.

TIME: O(n).

MANACHER

WHEN:

• Longest/count palindrome efficiently.

• Need palindrome radius.

TIME: O(n).

ROLLING HASH

WHEN:

• Compare substrings quickly.

• Detect equal substrings.

• Palindrome/hash matching.

• Use double hashing when collision risk matters.

SUFFIX ARRAY + LCP

WHEN:

• Lexicographic order of suffixes.

• Number/distinct substrings.

• Longest common substring/suffix-related problems.

• Advanced string problems.

SUFFIX AUTOMATON

WHEN:

• Many substring queries.

• Distinct substrings.

• Occurrence/frequency of substrings.

• Advanced string processing.

AHO-CORASICK

WHEN:

• Search MANY patterns simultaneously in one text.

• Multi-pattern matching.

10. DSU / DISJOINT SET UNION

WHEN:

• Dynamic merging of connected components.

• Connectivity queries.

• Kruskal MST.

• Detect cycle in undirected graph.

```
struct DSU{
    vector<int> p,sz;
    DSU(int n):p(n),sz(n,1){iota(p.begin(),p.end(),0);}
    int find(int x){return p[x]==x?x:p[x]=find(p[x]);}
    bool unite(int a,int b){
        a=find(a); b=find(b);
        if(a==b) return false;
        if(sz[a]<sz[b]) swap(a,b);
        p[b]=a; sz[a]+=sz[b];
        return true;
    }
};
```

11. FENWICK TREE / BIT

WHEN:

• Point update + prefix/range sum.

• Frequency queries.

• Inversion counting after coordinate compression.

• Need simpler/faster structure than segment tree.

TIME: O(log n).

```
struct Fenwick{
    int n; vector<ll> bit;
    Fenwick(int n):n(n),bit(n+1){}
    void add(int i,ll v){for(;i<=n;i+=i&-i)bit[i]+=v;}
    ll sum(int i){ll s=0;for(;i;i-=i&-i)s+=bit[i];return s;}
    ll rangeSum(int l,int r){return sum(r)-sum(l-1);}
};
```

12. SPARSE TABLE

WHEN:

• Array is STATIC.

• No updates.

• Many range min/max/GCD queries.

TIME: O(n log n) build, O(1) query for idempotent operations.

13. SEGMENT TREE

WHEN:

• Range query + point/range update.

• Sum/min/max/GCD/XOR.

• Need flexible associative operation.

You already have a tested template.

LAZY SEGMENT TREE

WHEN:

• Range update + range query.

• Example: add x to every element in [l,r] and query range sum.

• More advanced than normal segment tree.

14. PBDS / ORDER STATISTICS TREE

WHEN:

• Dynamic kth smallest.

• Count elements smaller than x.

• Ordered set with order statistics.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template<class T>
using ordered_set=tree<T,null_type,less<T>,rb_tree_tag,
                       tree_order_statistics_node_update>;
```

15. COORDINATE COMPRESSION

WHEN:

• Values up to 1e9/1e18 but only ordering matters.

• Need Fenwick/segment tree indexed by values.

```
vector<ll> v=a;
sort(v.begin(),v.end());
v.erase(unique(v.begin(),v.end()),v.end());
for(ll &x:a)
    x=lower_bound(v.begin(),v.end(),x)-v.begin();
```

16. SQRT DECOMPOSITION

WHEN:

• Need range queries/updates.

• Constraints are too small for complicated structures.

• Useful middle ground between brute force and segment tree.

17. MO'S ALGORITHM

WHEN:

• MANY offline range queries.

• Array does not change, or updates are handled with advanced Mo's.

• Need maintain a property while moving [l,r].

TIME: Usually around O((n+q)sqrt(n)) depending on operation.

Use when:

• Query order can be rearranged.

• Add/remove one element can be done efficiently.

18. GRAPH: BFS

WHEN:

• Unweighted graph.

• Minimum number of edges from source.

• Grid shortest path with equal movement cost.

TIME: O(V+E).

```
queue<int> q;
vector<int> dist(n,-1);
dist[s]=0;
q.push(s);
while(!q.empty()){
    int u=q.front(); q.pop();
    for(int v:g[u]){
        if(dist[v]==-1){
            dist[v]=dist[u]+1;
            q.push(v);
        }
    }
}
```

MULTI-SOURCE BFS

WHEN:

• Several starting nodes have distance 0.

• Nearest special node / infection / fire / monsters / facilities.

19. GRAPH: DFS

WHEN:

• Traversal.

• Connected components.

• Cycle detection.

• Tree problems.

• Component/subtree calculations.

```
void dfs(int u,int p){
    vis[u]=1;
    for(int v:g[u])
        if(v!=p && !vis[v]) dfs(v,u);
}
```

20. 0-1 BFS

WHEN:

• Edge weights are ONLY 0 or 1.

• Shortest paths.

TIME: O(V+E).

Use deque instead of priority_queue.

21. DIJKSTRA

WHEN:

• Single-source shortest path.

• ALL edge weights >= 0.

• Weighted graph.

TIME: O((V+E)logV) with binary heap.

```
vector<ll> d(n,INF);
priority_queue<pair<ll,int>,vector<pair<ll,int>>,
               greater<pair<ll,int>>> pq;
d[s]=0; pq.push({0,s});

while(!pq.empty()){
    auto [du,u]=pq.top(); pq.pop();
    if(du!=d[u]) continue;
    for(auto [v,w]:g[u]){
        if(d[v]>du+w){
            d[v]=du+w;
            pq.push({d[v],v});
        }
    }
}
```

22. BELLMAN-FORD

WHEN:

• Negative edge weights exist.

• Need detect reachable negative cycle.

TIME: O(VE).

Slower than Dijkstra.

23. FLOYD-WARSHALL

WHEN:

• All-pairs shortest paths.

• N is small, typically <= 400 depending on time limit.

• Dense graph.

TIME: O(n^3).

```
for(int k=0;k<n;k++)
 for(int i=0;i<n;i++)
  for(int j=0;j<n;j++)
   d[i][j]=min(d[i][j],d[i][k]+d[k][j]);
```

24. TOPOLOGICAL SORT

WHEN:

• Directed acyclic graph (DAG).

• Dependency ordering.

• Course/task scheduling.

• DP on DAG.

KAHN'S ALGORITHM:

• BFS using indegree.

• If processed vertices < n, graph has a cycle.

DFS TOPOLOGICAL SORT:

• DFS + postorder.

• Useful when already using DFS.

25. DAG DP

WHEN:

• Directed acyclic graph.

• Longest/shortest path or counting ways on DAG.

• Process nodes in topological order.

26. BIPARTITE CHECK

WHEN:

• Need 2-coloring.

• Detect odd cycle.

• Partition graph into two sets.

```
vector<int> col(n,-1);
queue<int> q;
col[s]=0;q.push(s);
while(!q.empty()){
    int u=q.front();q.pop();
    for(int v:g[u]){
        if(col[v]==-1) col[v]=col[u]^1,q.push(v);
        else if(col[v]==col[u]) /* not bipartite */;
    }
}
```

27. MST - KRUSKAL

WHEN:

• Need minimum spanning tree.

• Edges are given explicitly.

• DSU fits naturally.

TIME: O(E log E).

Steps:

1. Sort edges by weight.

2. Add edge if endpoints are in different DSU components.

28. MST - PRIM

WHEN:

• Need MST.

• Graph is adjacency-list oriented.

• Repeatedly take cheapest edge connecting tree to outside.

• Priority queue implementation.

TIME: O(E log V).

29. SCC - KOSARAJU

WHEN:

• Directed graph.

• Need strongly connected components.

• Condense graph into DAG.

TIME: O(V+E).

30. SCC - TARJAN

WHEN:

• Same SCC problem as Kosaraju.

• Want one DFS-based algorithm.

TIME: O(V+E).

31. BRIDGES

WHEN:

• Undirected graph.

• Find edges whose removal increases components.

• Network vulnerability.

• Edge-biconnected structure.

TIME: O(V+E).

32. ARTICULATION POINTS

WHEN:

• Undirected graph.

• Find vertices whose removal disconnects the graph.

TIME: O(V+E).

33. LCA / BINARY LIFTING

WHEN:

• Tree.

• Many Lowest Common Ancestor queries.

• kth ancestor.

• Distance between nodes.

TIME: O(log n) per query after O(n log n) preprocessing.

Distance:

dist(u,v)=depth[u]+depth[v]-2*depth[lca(u,v)].

34. TREE DIAMETER

WHEN:

• Need longest path in a tree.

• Tree diameter / eccentricity-related problems.

Standard:

1. BFS/DFS from any node -> farthest A.

2. BFS/DFS from A -> farthest B.

3. distance(A,B) is diameter.

TIME: O(V).

35. EULER TOUR OF TREE

WHEN:

• Convert subtree queries into array ranges.

• Need subtree sum/query with Fenwick/segment tree.

• Entry/exit times.

Key property:

subtree(v) becomes a contiguous interval in Euler order.

36. HEAVY-LIGHT DECOMPOSITION

WHEN:

• MANY path queries on a tree.

• Path sum/min/max + updates.

• Need segment-tree power over tree paths.

TIME: Usually O(log^2 n) per path query/update.

37. MAX FLOW

WHEN:

• Network capacity.

• Maximum amount from source to sink.

• Bipartite matching can be modeled as flow.

• Cut/capacity problems.

Algorithms:

• Dinic: standard CP choice.

• Edmonds-Karp: simple but slower.

38. 2-SAT

WHEN:

• Boolean variables.

• Constraints are clauses of two literals:

(a OR b).

• Need determine satisfiability and possibly assignment.

Use implication graph + SCC.

39. EULERIAN PATH / CIRCUIT

WHEN:

• Need traverse EVERY EDGE exactly once.

• Undirected/directed graph.

• Euler path/circuit construction.

Do NOT confuse with Hamiltonian path:

Euler = every EDGE.

Hamiltonian = every VERTEX.

40. BINARY LIFTING BEYOND LCA

WHEN:

• kth ancestor.

• Jump k steps in a tree/functional graph.

• Binary lifting on parent transitions.

TIME: O(log n) per jump.

41. FUNCTIONAL GRAPH

WHEN:

• Every node has exactly one outgoing edge.

• Find cycles, distance to cycle, kth successor.

Useful tools:

• Binary lifting.

• Cycle detection.

• In-degree peeling.

42. ADVANCED DATA STRUCTURES

LAZY SEGMENT TREE

• Range update + range query.

PERSISTENT SEGMENT TREE

WHEN:

• Need versions of an array.

• Query historical versions.

• kth number in ranges in advanced problems.

TREAP

WHEN:

• Need randomized balanced BST.

• Split/merge sequences.

• Implicit treap for sequence operations.

DSU ON TREE

WHEN:

• Tree subtree queries.

• Need frequency/distinct/color statistics.

• Offline-ish "small-to-large" subtree processing.

LI CHAO TREE

WHEN:

• Maintain lines y=mx+b.

• Query minimum/maximum at x.

• Dynamic convex hull trick with arbitrary line insertion.

CONVEX HULL TRICK

WHEN:

• DP transitions contain min/max of lines.

• Query x against a set of linear functions.

• Slopes/queries have useful ordering for simpler deque version.

43. NUMBER THEORY

GCD

WHEN:

• Divisibility, common factors, simplifying ratios.

```
ll g=gcd(a,b);
```

LCM

WHEN:

• Common multiple / periodic synchronization.

```
ll lcmll(ll a,ll b){return a/gcd(a,b)*b;}
```

BINARY EXPONENTIATION

WHEN:

• a^b.

• Modular exponentiation.

TIME: O(log b).

```
ll binpow(ll a,ll b,ll mod){
    ll r=1;
    while(b){
        if(b&1) r=r*a%mod;
        a=a*a%mod;
        b>>=1;
    }
    return r;
}
```

EXTENDED GCD

WHEN:

• ax+by=gcd(a,b).

• Modular inverse for non-prime modulus.

• Linear Diophantine equations.

SIEVE OF ERATOSTHENES

WHEN:

• Need all primes <= N.

• Many prime queries.

TIME: O(n log log n).

SPF / SMALLEST PRIME FACTOR

WHEN:

• Many factorizations for numbers <= N.

• Divisor/factor based queries.

TIME: O(n log log n) build, very fast factorization.

TRIAL DIVISION FACTORIZATION

WHEN:

• Only a few numbers need factorization.

• Factor one large number by sqrt(n).

NUMBER OF DIVISORS

WHEN:

• Need count of divisors.

If n=p1^a1*p2^a2...:

answer=(a1+1)(a2+1)...

EULER PHI

WHEN:

• Count numbers <=n coprime to n.

• Euler theorem / multiplicative number theory.

MOBIUS FUNCTION

WHEN:

• Inclusion-exclusion over divisors.

• Count coprime pairs.

• Advanced number-theory transforms.

DIVISOR SIEVE

WHEN:

• Need all divisors/frequency contributions for every number <= N.

44. MODULAR ARITHMETIC

FERMAT LITTLE THEOREM

WHEN:

• MOD is prime.

• Need modular inverse:

a^(MOD-2) mod MOD.

MODULAR INVERSE

WHEN:

• Need division under modulo.

• gcd(a,MOD)=1.

nCr MOD PRIME

WHEN:

• Many combinations.

• n <= precomputation limit.

Use factorial + inverse factorial.

LUCAS THEOREM

WHEN:

• n,r are huge relative to prime modulus.

• Compute C(n,r) mod prime using base-p digits.

CHINESE REMAINDER THEOREM

WHEN:

• Need solve simultaneous congruences with compatible moduli.

• Typical form x=a1 mod m1, x=a2 mod m2,...

LINEAR DIOPHANTINE

WHEN:

• Equation ax+by=c.

• Need integer solutions.

45. BIT MANIPULATION

CHECK BIT

```
if(x&(1LL<<i)){}
```

SET BIT

```
x|=(1LL<<i);
```

CLEAR BIT

```
x&=~(1LL<<i);
```

TOGGLE

```
x^=(1LL<<i);
```

LOWEST SET BIT

```
x&-x
```

REMOVE LOWEST SET BIT

```
x&=x-1;
```

COUNT BITS

```
__builtin_popcount(x);
__builtin_popcountll(x);
```

WHEN:

• XOR problems.

• Subsets.

• Bitmask DP.

• Powers of two.

• Compact state representation.

SUBMASK ENUMERATION

WHEN:

• Enumerate all subsets of mask.

```
for(int sub=mask;sub;sub=(sub-1)&mask){}
```

46. DP PATTERNS

1D DP

WHEN:

• State depends on previous few positions/states.

KNAPSACK 0/1

WHEN:

• Each item can be used at most once.

• Capacity/sum constraint.

UNBOUNDED KNAPSACK

WHEN:

• Items can be reused unlimited times.

LIS

WHEN:

• Longest increasing subsequence.

• O(n log n) patience sorting when only length is required.

LCS

WHEN:

• Longest common subsequence of two strings/sequences.

• Usually O(nm).

INTERVAL DP

WHEN:

• State is an interval [l,r].

• Matrix chain, merging, removing ends, palindrome-like interval processes.

BITMASK DP

WHEN:

• n is small, typically <= 20-22.

• State is a subset of items.

DIGIT DP

WHEN:

• Count numbers <= N satisfying digit constraints.

• N is huge as a string.

• Constraints depend on digits/leading zeros/tightness.

TREE DP

WHEN:

• Answer depends on subtrees.

• Root tree and combine child states.

DP ON DAG

WHEN:

• Dependencies form a DAG.

• Longest/shortest path or count ways.

47. MATRIX EXPONENTIATION / FAST DOUBLING

MATRIX EXPONENTIATION

WHEN:

• Linear recurrence.

• nth Fibonacci/sequence state.

• n is huge.

TIME: O(k^3 log n) for k x k matrix.

FAST DOUBLING FIBONACCI

WHEN:

• Only Fibonacci number is needed.

TIME: O(log n).

48. GEOMETRY BASICS

CROSS PRODUCT

WHEN:

• Orientation of three points.

• Collinearity.

• Segment intersection.

• Convex hull.

```
ll cross(pair<ll,ll> a,pair<ll,ll> b,pair<ll,ll> c){
    return (b.first-a.first)*(c.second-a.second)
         - (b.second-a.second)*(c.first-a.first);
}
```

DOT PRODUCT

WHEN:

• Angle/projection/perpendicularity checks.

DISTANCE SQUARED

WHEN:

• Compare distances without floating point.

CONVEX HULL

WHEN:

• Smallest convex polygon containing points.

• Extreme boundary / geometry optimization.

• Monotonic chain is standard.

49. ADVANCED PRIMES / FACTORIZATION

MILLER-RABIN

WHEN:

• Need primality testing for very large 64-bit integers.

• Much faster than sqrt(n).

POLLARD RHO

WHEN:

• Factor very large 64-bit integers.

• Advanced number theory.

FFT / NTT

WHEN:

• Polynomial multiplication.

• Convolution.

• Huge combinational/convolution problems.

• NTT when modulus supports suitable roots of unity.

50. COMMON PATTERN: PREFIX + SUFFIX ANSWER

WHEN:

• Try every split i.

• Left answer can be precomputed from left.

• Right answer can be precomputed from right.

```
vector<ll> prefAns(n),suffAns(n);

for(int i=0;i<n;i++){
    // best answer for [0..i]
}

for(int i=n-1;i>=0;i--){
    // best answer for [i..n-1]
}

for(int i=0;i+1<n;i++){
    // combine prefAns[i] + suffAns[i+1]
}
```

51. PROBLEM -> ALGORITHM CHEAT SHEET

"Many range sums"

-> Prefix Sum.

"Many range additions, final array"

-> Difference Array.

"Static range min/max/GCD"

-> Sparse Table.

"Point update + range sum"

-> Fenwick.

"Complex range query + update"

-> Segment Tree.

"Range update + range query"

-> Lazy Segment Tree.

"Longest/shortest valid subarray"

-> Sliding Window.

"Sorted array + pair condition"

-> Two Pointers.

"Next greater/smaller"

-> Monotonic Stack.

"Window max/min"

-> Monotonic Queue.

"Maximum subarray sum"

-> Kadane.

"Find element in sorted array"

-> Binary Search.

"Minimum possible answer / maximum possible answer"

-> Binary Search on Answer.

"Top k elements"

-> Heap / nth_element.

"Repeatedly take minimum"

-> Min Heap.

"Repeatedly take maximum"

-> Max Heap.

"Dynamic median"

-> Two Heaps.

"Unweighted shortest path"

-> BFS.

"0/1 edge weights"

-> 0-1 BFS.

"Non-negative weighted shortest path"

-> Dijkstra.

"Negative edges"

-> Bellman-Ford.

"All-pairs shortest path, small n"

-> Floyd-Warshall.

"Dependencies / DAG ordering"

-> Topological Sort.

"Shortest/longest path in DAG"

-> Topological DP.

"Connect components dynamically"

-> DSU.

"Minimum spanning tree"

-> Kruskal / Prim.

"Directed strongly connected components"

-> SCC.

"Critical edge"

-> Bridge.

"Critical vertex"

-> Articulation Point.

"Tree ancestor / LCA"

-> Binary Lifting / LCA.

"Many tree path queries"

-> HLD + Segment Tree.

"Many subtree queries"

-> Euler Tour + Fenwick/Segment Tree.

"Every edge exactly once"

-> Eulerian Path/Circuit.

"Every vertex exactly once"

-> Hamiltonian (usually exponential/NP-hard; small n only).

"Prefix of words"

-> Trie.

"Maximum XOR"

-> Binary Trie.

"Pattern matching"

-> KMP / Z.

"Longest palindrome"

-> Manacher.

"Compare substrings"

-> Rolling Hash.

"Many patterns in one text"

-> Aho-Corasick.

"Suffix ordering / substring complexity"

-> Suffix Array + LCP.

"Dynamic kth smallest"

-> PBDS / Fenwick with compression / Segment Tree.

"Huge values, only relative order matters"

-> Coordinate Compression.

"Many offline range queries"

-> Mo's Algorithm.

"Tree subtree frequency/color queries"

-> DSU on Tree.

"Historical versions"

-> Persistent Segment Tree.

"Maintain lines, query min/max"

-> Convex Hull Trick / Li Chao Tree.

"All subsets, n <= ~20"

-> Bitmask DP.

"Count numbers <= N with digit restrictions"

-> Digit DP.

"Linear recurrence with huge n"

-> Matrix Exponentiation / Fast Doubling.

"Many primes"

-> Sieve.

"Many factorisations <= N"

-> SPF.

"One/few factorisations"

-> Trial Division.

"Combinations modulo prime"

-> Factorials + Inverses.

"Simultaneous modular equations"

-> CRT.

"Very large 64-bit prime test"

-> Miller-Rabin.

"Very large 64-bit factorization"

-> Pollard Rho.

"Polynomial multiplication"

-> FFT / NTT.

52. COMPLEXITY CHEAT SHEET

O(1):

• Prefix/range query after preprocessing

• Sparse Table query

• Basic hash average lookup

O(log n):

• Binary Search

• Fenwick

• Segment Tree

• Heap operation

• PBDS

• Fast Power

• LCA after preprocessing

O(n):

• Prefix/Suffix

• Sliding Window

• Two Pointers

• Kadane

• Monotonic Stack

• Monotonic Queue

• BFS/DFS: O(V+E)

• KMP/Z/Manacher

O(n log n):

• Sorting

• DSU operations overall near linear

• Coordinate Compression

• Dijkstra with binary heap: O((V+E)logV)

• Kruskal: O(E log E)

• Sparse Table build

O(n^2):

• Basic LCS

• Some DP

• Dense graph algorithms depending on n

O(n^3):

• Floyd-Warshall

• Matrix multiplication per exponentiation step

O(2^n):

• Subset enumeration

• Bitmask brute force

O(3^n):

• Some submask-partition DP.

53. MUST KNOW COLD

1. Prefix/Suffix

2. Difference Array

3. Sliding Window

4. Two Pointers

5. Binary Search

6. Binary Search on Answer

7. Stack/Queue/Deque

8. Monotonic Stack

9. Monotonic Queue

10. Kadane

11. Sorting + custom comparator

12. Heap / Priority Queue

13. BFS

14. DFS

15. Dijkstra

16. Topological Sort

17. DSU

18. Kruskal

19. Fenwick

20. Segment Tree

21. GCD/LCM

22. Fast Power

23. Sieve

24. Prime Factorization

25. SPF

26. Bit Manipulation

27. Coordinate Compression

28. Trie

29. KMP

30. Z Algorithm

54. SHOULD KNOW

- Sparse Table
- PBDS
- Binary Trie
- nCr
- Euler Phi
- Extended GCD
- Matrix Exponentiation
- Manacher
- Rolling Hash
- Bellman-Ford
- 0-1 BFS
- Floyd-Warshall
- SCC
- Bridges
- Articulation Points
- LCA
- Euler Tour
- Tree Diameter
- Prim
- Dinic basics
- Mo's Algorithm

55. ADVANCED

- Lazy Segment Tree
- HLD
- DSU on Tree
- Persistent Segment Tree
- Treap
- Li Chao Tree
- Convex Hull Trick
- Aho-Corasick
- Suffix Array
- Suffix Automaton
- 2-SAT
- Miller-Rabin

• Pollard Rho

• FFT/NTT

• Lucas Theorem

• CRT

• Mobius

• Advanced digit DP

56. FINAL RULE

Before choosing an algorithm, ask:

1. What is n?

2. What is the time limit?

3. Is the data static or dynamic?

4. Is the array sorted?

5. Is the condition monotonic?

6. Are graph edges weighted?

7. Can weights be negative?

8. Is the graph directed?

9. Is it a tree/DAG?

10. Are queries online or offline?

11. Can I reorder queries?

12. Is there a prefix/suffix decomposition?

13. Is there a two-pointer/sliding-window invariant?

14. Is the operation associative?

15. Is there a monotonic stack/queue pattern?

16. Is the answer itself binary-searchable?

17. Are values huge but only their ordering matters?

18. Is n small enough for bitmask DP?

19. Does the problem involve divisibility/primes/modulo?

20. What is the simplest algorithm that fits the constraints?

THE GOAL:

Do not learn 100 algorithms and randomly throw them at problems.

Learn to recognize patterns.

Pattern recognition + correct invariant + complexity awareness

is what turns templates into actual CP skill.